通用人工智能创新实践课 - Agent 基本概念

课程介绍

Lesson1： Agent 基本概念介绍演进脉络，分类，构造你的第一个智能体

Lesson2：前 Agent 时期主流技术： Propmt Engineering ， CoT ， RAG 等

Lesson3： LLM 模型 API 接口介绍、主流框架比对和介绍， TongAgents SDK

Lesson4：工具使用： MCP 、 CLI 、 Skill 、 Sandbox

Lesson5： Agent 经典构建范式： ReAct 、 Plan-and-Solve 、 Reflection 、 Loop Engineering

Lesson6：记忆与检索： RAG 详解、记忆分类、存储检索机制

Lesson7：多智能体框架， A2A/ANP

Lesson8：智能体自演进

GitHub - temp-bigai/tongagents-course · GitHub

推荐学习

HelloAgents https://datawhalechina.github.io/hello-agents/#/

Microsoft https://github.com/microsoft/ai-agents-for-beginners

Hugging Face https://huggingface.co/learn/agents-course

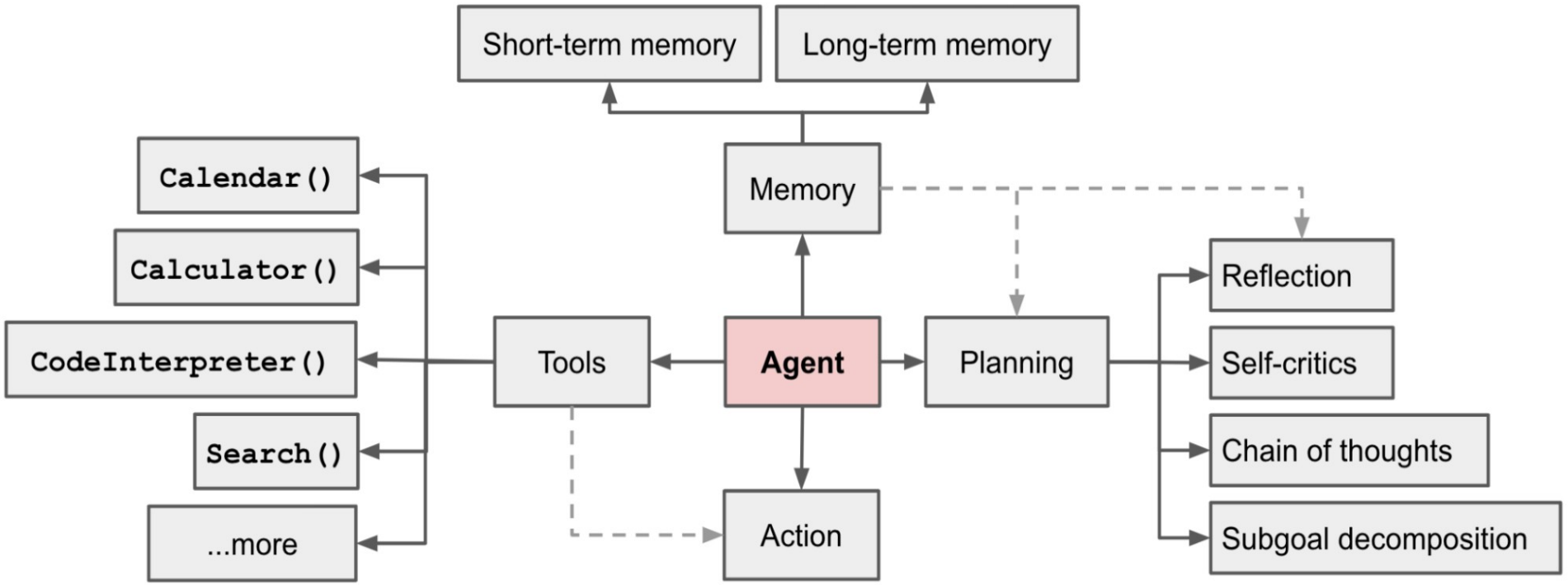
CMU https://www.cmu-agents.com/

Standford https://cs329z.stanford.edu/

吴恩达 Agentic AI https://www.deeplearning.ai/courses/agentic-ai

Agent 定义

任何能够通过传感器感知其所处环境，并自主地通过执行器采取行动以达成特定目标的实体



LLM Agent 定义 From https://lilianweng.github.io/posts/2023-06-23-agent/

Agent 演进

早期智能体

- 1966 Eliza 对话机器人
- 1972 Shakey 移动机器人
- 1997 国际象棋“深蓝”
- 2011 IBM Watson
- 2016 Alpha go

狭义智能体，场景受限

大模型智能体

- 基于基础大模型构建
- propmt、CoT、RAG
- chatbot、coding copilot、content generator

基于大模型能力的扩展

Agentic 智能体

- 生成式 AI 演进为智能体 AI
- plan + memory + tool
- Context Engineering、Harness Engineering、Loop Engineering

智能体框架迭代演进

早期 AI 技术

- 1960s 符号主义 AI
- 1970s 专家系统
- 1990s 机器学习
- 2010s 深度学习

2023 GPT

- 自然语言处理能力突破
- 推理能力泛化性提升
- 丰富的知识储备

2025 DeepSeek

- 低成本，降低硬件门槛。
- 性能好，具备良好的推理能力和效率。
- 开源，AI 技术平权

Agent 分类 - 基于决策架构

反应式智能体 (Reactive Agents)

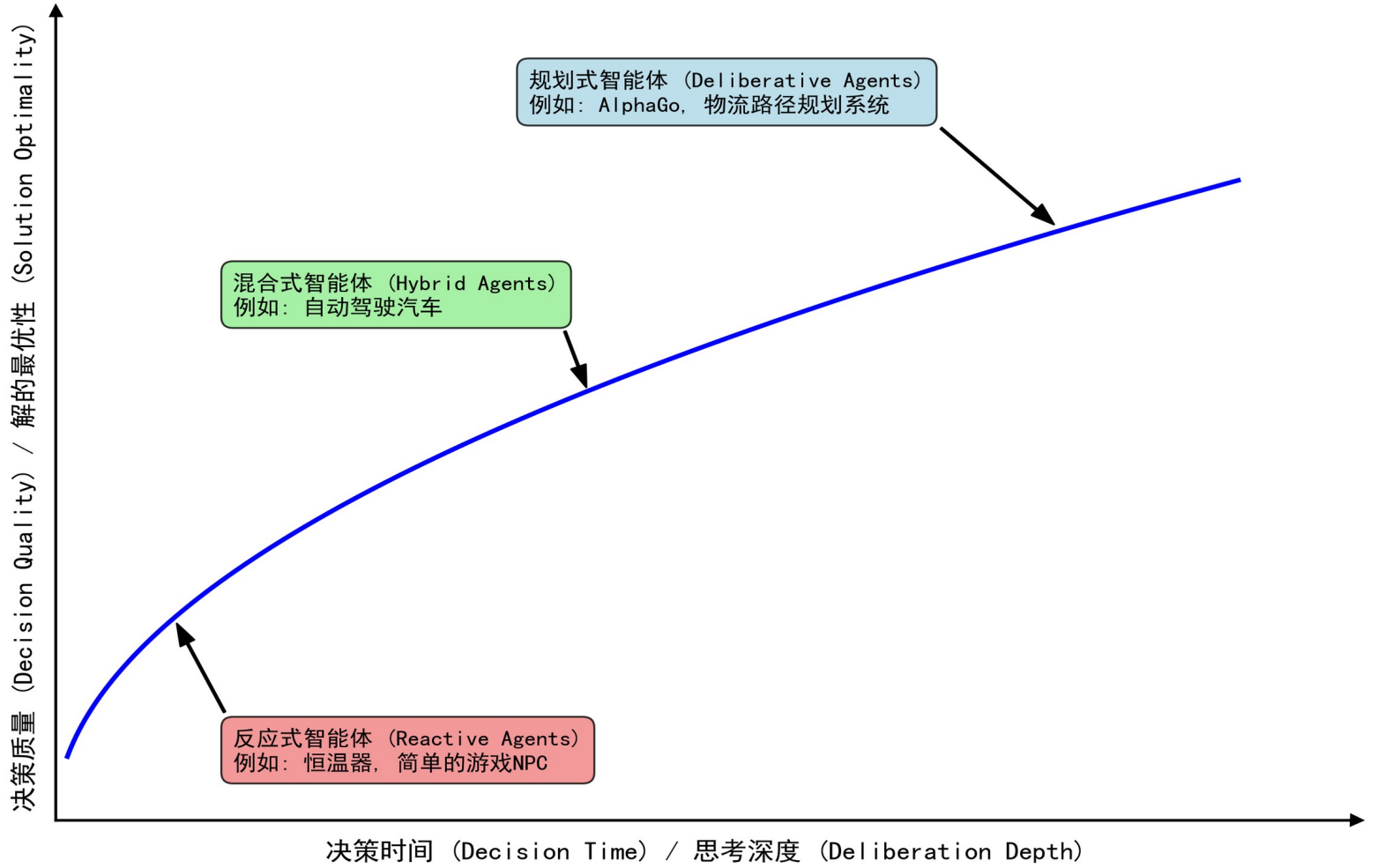
- 感知到行动的直接映射
- 速度快、计算开销低

规划式智能体 (Deliberative Agents)

- 探索各种可能性，评估行动序列的后果，找到达成目标的最佳路径
- 基于目标和基于效用

混合式智能体 (Hybrid Agents)

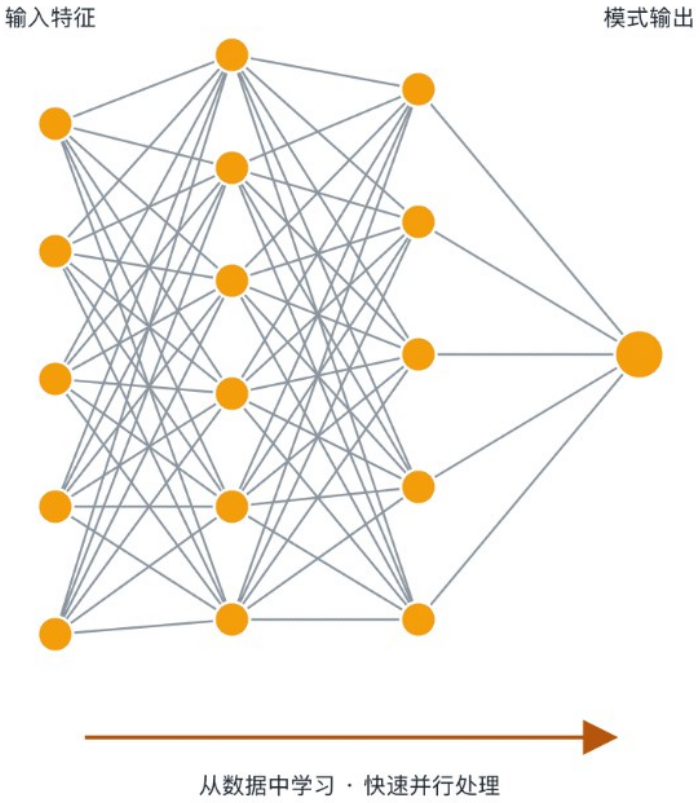
- 分层设计：即时反应 + 长远规划



Agent 分类 - 基于知识表法

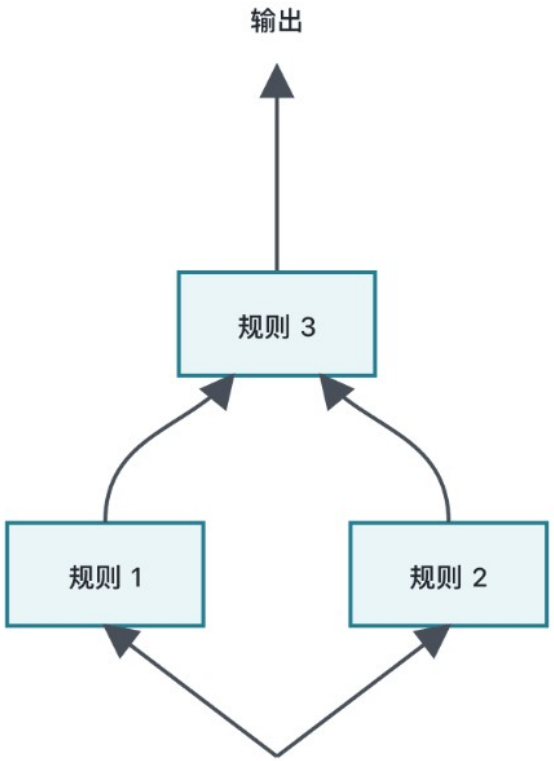
连接主义 AI（Connectionism AI）

模式识别 · 系统 1



符号主义 AI（Symbolic AI）

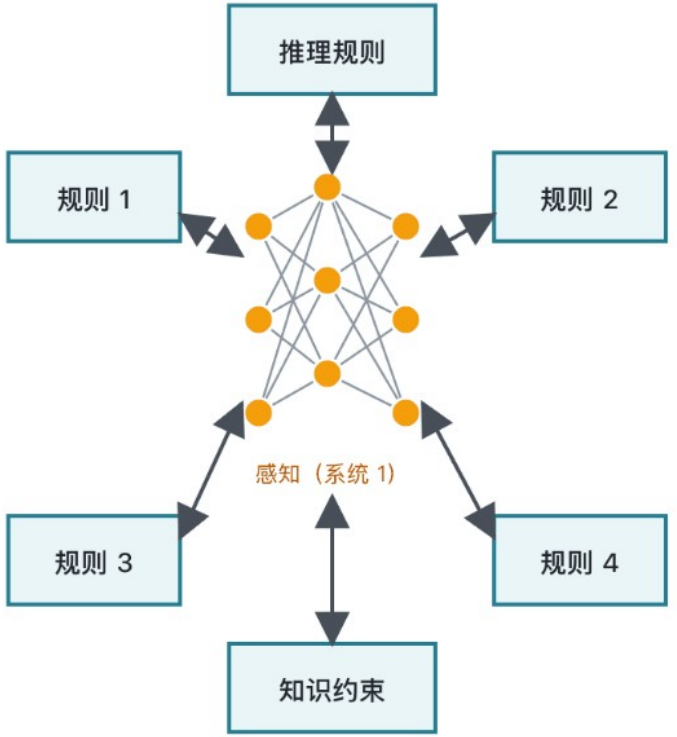
逻辑推理 · 系统 2



显式规则 · 可追溯推理

神经符号主义 AI（Neuro-symbolic AI）

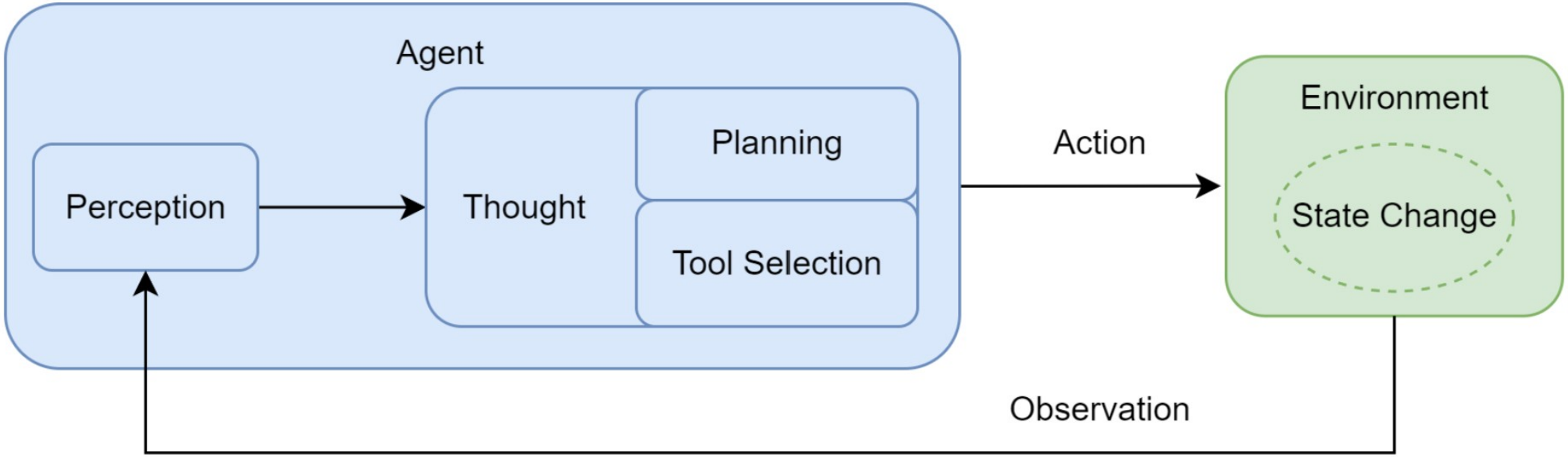
感知与推理协同 · 系统 1 + 系统 2



符号约束与推理（系统 2）

神经网络与符号系统双向协作

Agent 基本运行机制



Perception → Planning → Action → Observation

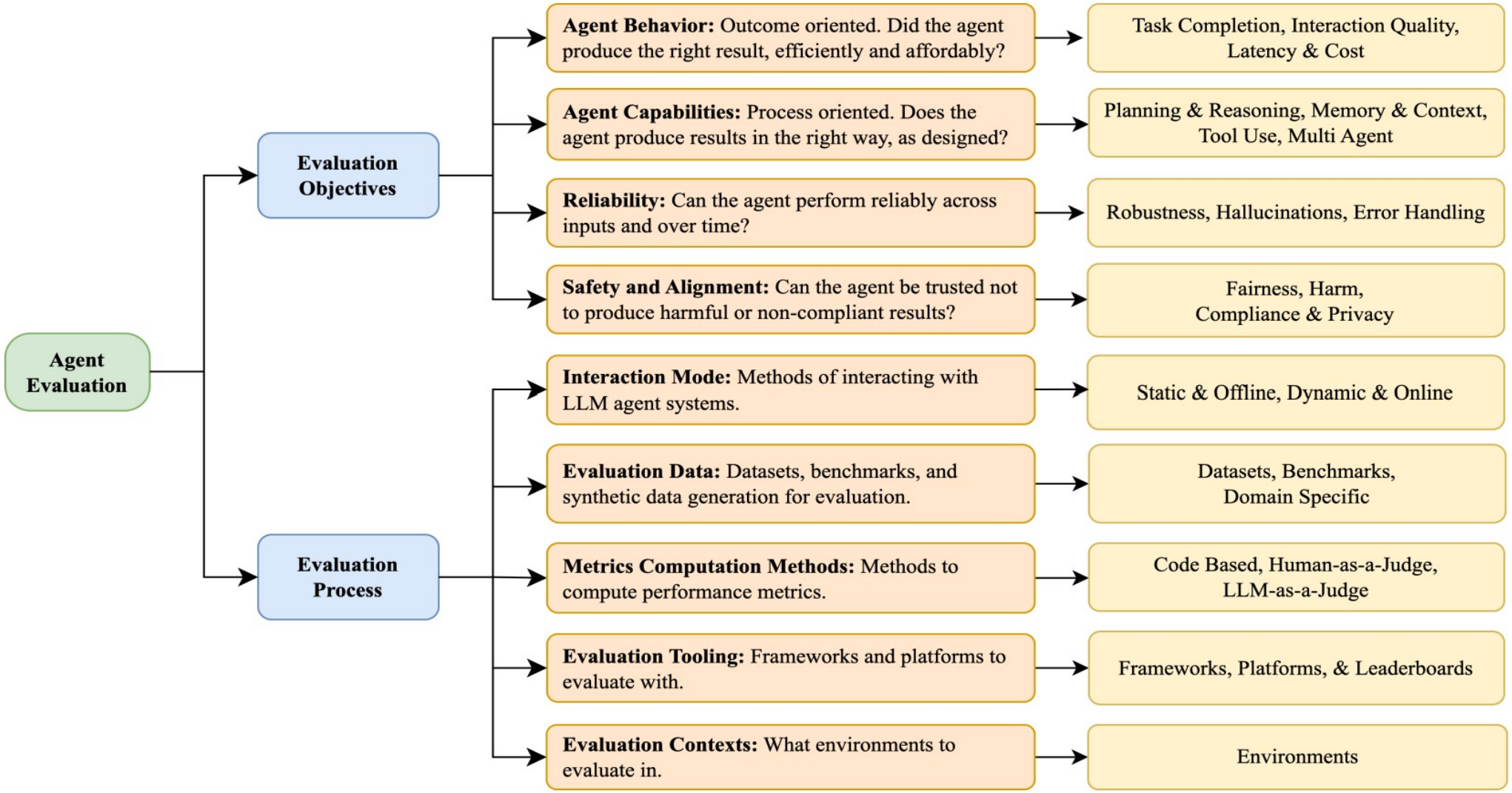
S Yao et al.2022, ReAct: Synergizing Reasoning and Acting in Language Models

L Wang et al.2023, Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by LLM

Shinn et al.2024, Reflexion: Language Agents with Verbal Reinforcement Learning

Anthropic 2026， Loop Engineering

Agent 评测方法


Agent Evaluation
Evaluation Objectives
Evaluation Process
Agent Behavior: Outcome oriented. Did the agent produce the right result, efficiently and affordably?
Task Completion, Interaction Quality, Latency & Cost
Agent Capabilities: Process oriented. Does the agent produce results in the right way, as designed?
Planning & Reasoning, Memory & Context, Tool Use, Multi Agent
Reliability: Can the agent perform reliably across inputs and over time?
Robustness, Hallucinations, Error Handling
Safety and Alignment: Can the agent be trusted not to produce harmful or non-compliant results?
Fairness, Harm, Compliance & Privacy
Interaction Mode: Methods of interacting with LLM agent systems.
Static & Offline, Dynamic & Online
Evaluation Data: Datasets, benchmarks, and synthetic data generation for evaluation.
Datasets, Benchmarks, Domain Specific
Metrics Computation Methods: Methods to compute performance metrics.
Code Based, Human-as-a-Judge, LLM-as-a-Judge
Evaluation Tooling: Frameworks and platforms to evaluate with.
Frameworks, Platforms, & Leaderboards
Evaluation Contexts: What environments to evaluate in.
Environments

Agent benchmark

方向	代表 Benchmark	核心功能	特点
通用 Agent	GAIA	多步推理 + Web + 文件 + 工具	最接近“通用数字助理”
软件工程	SWE-bench	修复真实 GitHub Issue	真实代码库、自动测试验收
Web Agent	WebArena	浏览网页、搜索、填写表单、跨页面操作	长链路 Browser Agent
Computer Use	OSWorld	操作真实桌面 GUI	鼠标 / 键盘 / 应用 / 跨软件工作流
Tool Use	τ-bench / τ^2-bench	调工具、遵守业务规则、与用户交互	特别强调“正确调用工具 + policy
Terminal Agent	Terminal-Bench	Linux Shell / CLI 多步任务	更接近真实开发 / 运维环境
长任务能力	METR Time Horizon / HCAST	Agent 能独立完成多长时间的人类工作	覆盖面广，适合做历史 / 横向比较
科研 / 复杂推理	RE-Bench 等	AI Research / ML Research 工作	衡量长周期研究型 Agent
End-to-End 工作	OneMillion-Bench/Vending-Bench	真实用户 / 商业工作流	从“ benchmark task” 转向“真正交付结果”

OneMillion-Bench: How Far are Language Agents from Human Experts?

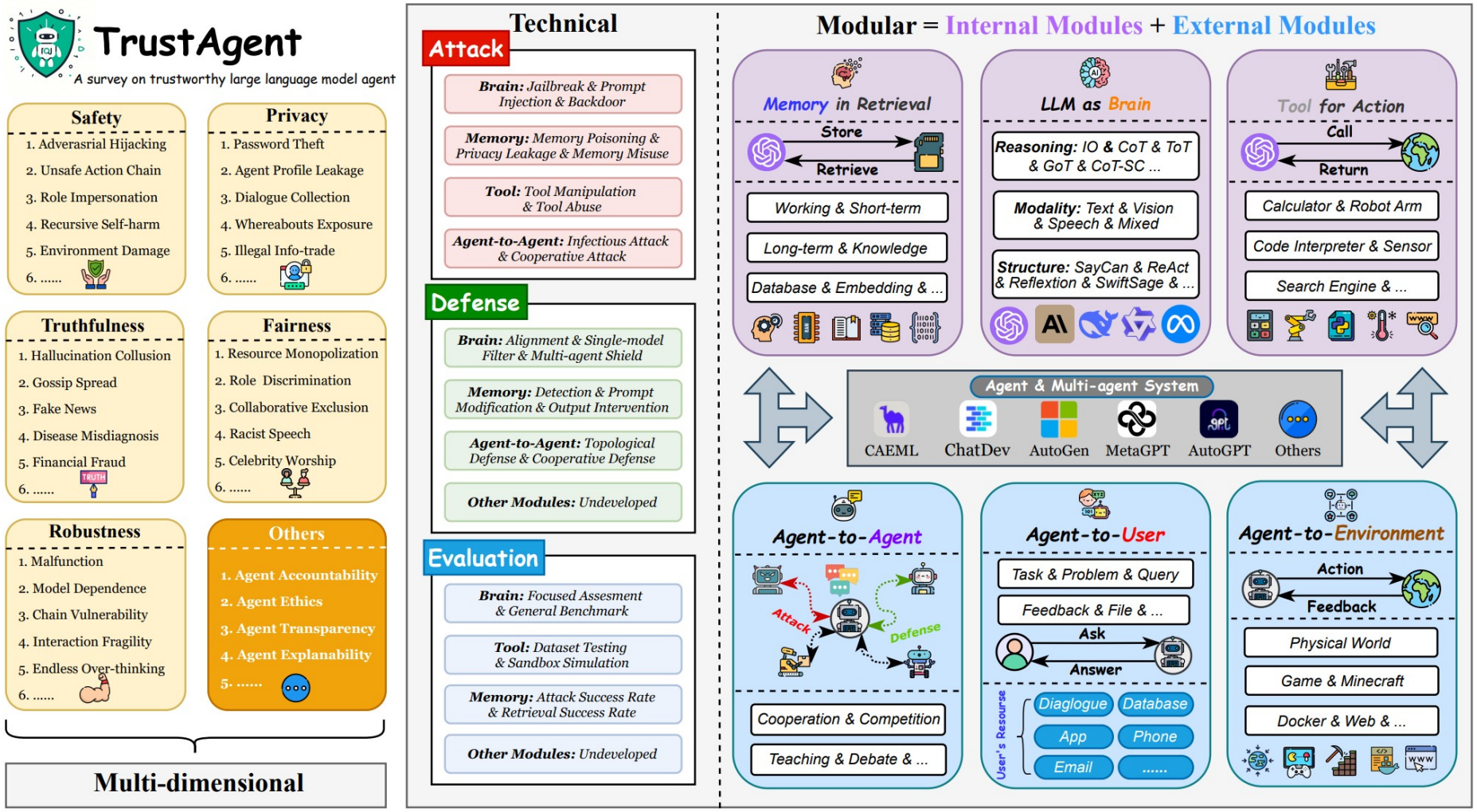
Agent 可信架构

攻击手段

- 提示词注入
- 工具误用
- 权限越界
- 记忆 / 知识投毒

防御体系

- 输入防护、输出干预
- 沙箱仿真
- 协作 / 拓扑防御



Y Miao et al. 2025， A Survey on Trustworthy LLM Agents: Threats and Countermeasures

TongAgents 介绍



虚拟数智人 **非具身智能体** **具身机器人**

快速低成本打造多种形态行业智能体

行业智能体基座： TongAgents

工具组件

模型管理			能力工具				虚拟形象		智能硬件		系统工具		
版本管理	训练微调	MaaS	计算机视觉	文本处理	语音对话	SKILLS MCP	形象库	形象复刻	IOT组件	ROS组件	云原生	部署运维	命令行操作

基础框架

Agent 框架							Env 框架				基础框架		
Agent 设定	知识记忆	价值能力	多智能体协作	A2A 协议支持	规划与执行	任务与子代理	配置系统	事件机制	日志轨迹	沙箱机制	运行策略	权限体系	审计追溯

TongAgents SDK + CLI

面向智能体开发与编排的开源 SDK 与交互式 Shell

https://tongagents-builder.mybigai.ac.cn/docs/Tong-Agent/

TongAgent SDK

可组合 SDK，内置 100+ 集成能力，可快速构建智能体、聊天机器人、 RAG 流水线和工具调用工作流。

Python 开发套件

+

TongAgent CLI

交互式 Shell，即时进行智能体实验、场景测试与生命周期管理。

Python CLI

SDK 核心组件：

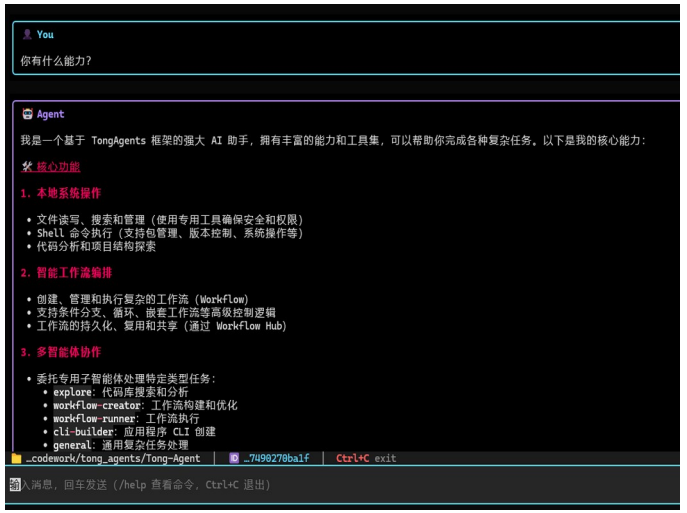
- 智能体
- 工具
- 记忆
- 工作流
- 多 LLM 支持
- 流式输出

```python
import os
from tongagents.agents.llm.base import ModelConfig
from tongagents.agents.llm_agent import (
    LLMRunContext,
    ReactAgent,
    ReactAgentSetting,
)

agent_settings = ReactAgentSetting(
    llm_config=ModelConfig(
        model_name=os.environ.get("MODEL_NAME"),
        url=os.environ.get("MODEL_URL")),
)

# 创建agent并配置
agent = ReactAgent(
    dep_context=LLMRunContext(),
    agent_settings=agent_settings,
)

result = agent.step("请输出Hello World")
print(''.join(msg.content for msg in result))
"""
Hello World
"""
```



TongAgents SDK 核心框架与组件介绍

4 大核心支柱

- **A** **可控性** 显式 DAG 工作流 + 隐式 ReAct 循环
- **B** **持久化** SQL/Redis 后端记忆，支持多智能体与容错
- **C** **工具调用** 100+ 集成：bash / read /write / glob / grep / search
- **D** **流式输出** EventBus 实时发布 token 、步骤与错误

Agent 核心循环

1. **开始** 用户查询
2. **思考** LLM 决策
3. **行动** 调用工具
4. **观察** 循环直至结束

PromptTemplate

LLMFactory
Workflow

ToolManager
MCP+Skill+Tool
Sandbox+Knowlege

Memory
Session Persist

Agent
Component

循环回 THINK，直至任务完成

追寻 | 大一统理论　创造 | 通用智能体　提升 | 全人类福祉

#	组件	职责	典型用法
1	Agent	智能体核心；4 抽象方法 run/step/arun/astep; ReAct 风格实现，支持工具 + 知识库	`agent = ReactAgent(dep_context=LLMRunContext(), agent_settings=...)` `result = agent.step("Hello World")`

2	工作流	DAG 编排，多步骤自动化；节点 + 边 + 上下文 + 追踪	`wf = Workflow()`
			`wf.add_node("step1", node_fn)`
			`wf.add_edge("step1", "step2")`

3	Runtime variable 与节点鉴权	节点间共享运行时变量 + 模板渲染 ({{runtime.TOKEN}}); 校验缺失 / 不支持变量	`# 注入：`
			`deps["runtime_variables"] = {"TOKEN": "xxx"}`
			`# 节点内：{{runtime.TOKEN}}`
			`自动渲染`

4	模型	LLM 抽象 + 多 provider 统一接入；LLM_FACTORY.create(config)	`cfg = ModelConfig(model_name="deepseek-chat", url="...")` `llm = LLM_FACTORY.create(cfg)`

5	工具	工具调用生态：函数→ Tool，注册到 Manager，远程 MCP 服务，OpenAPI 自动转换	`@tool()`
			`def weather_tool(location, time): ...`
			`svc = ToolService(); tool = svc.get_tool("weather_tool")`

6	知识库	RAG 文档检索；抽象 add_documents/search/delete/clear/stats; 多向量库后端	`store = ChromaVectorStore()`
			`store.add_documents(chunks)`
			`results = store.search(q, limit=3)`

7	记忆	短期 / 长期记忆；add_message/add_chunk/get_messages/clear; SQL 与 Redis 双后端	`memory = Memory()`
			`memory.add_message(UserPromptMessage(content="hi"))`
			`history = memory.get_messages(limit=50)`

8	数据处理	数据预处理全链路：多格式加载 → 切块 → 增强；喂给 KnowledgeStore / Agent	`docs = DirectoryLoader(input_files=["./kb.txt"]).load()` `chunks = default_text_splitter.split_document(docs)`

课后习题

作业目标

在 githup 的 echo agent 基础上，给 Agent 添加 4 个 OS 操作工具，从「只会回显」升级为「能干活」

核心任务

在 echo agent 基础上，编写 read-file， write-file， edit-file， bash 4 个工具实现。
并将其注册到 Agent 中，可以让 agent 驱动这些工具来完成用户需求。

关键要求（提示）

✓ 可以用 @tool() 装饰器

✓ demo 端到端跑通

最终效果

调用 Agent，输入 query，帮我看下当前目录下的 xxxx 文件， agent 可以调用 read-file 工具来读取，告知你文件内容

学完你将掌握

1. @tool() 装饰器： docstring → schema
2. ReactAgent(tools=[...]) 注入